Developer Metrics in the AI Era

Meltano + DuckDB + Elasticsearch

DT Mirizzi ▪ April 2026

About Me

DT Mirizzi

- Principal Software Engineer @ **Palo Alto Networks**
- Previously **Obsidian Security**
- Maintainer of `target-elasticsearch` for Meltano

Five Years Ago…

I gave a talk on **Meltano + Elasticsearch** for developer metrics.

I built `target-elasticsearch` for Meltano. (still maintaining it)

The why: We saw a massive decline in productivity across a 200-engineer organization. Leadership wanted answers. We needed to collect the *why*.

SPACE and DORA metrics told us most of what we wanted.

SPACE & DORA

DORA — DevOps Research & Assessment

- **Deployment Frequency** — how often you ship
- **Lead Time for Changes** — commit to production
- **Mean Time to Recovery** — incident response
- **Change Failure Rate** — % of deploys causing failures

SPACE — Developer Productivity

- **S**atisfaction & Well-being
- **P**erformance
- **A**ctivity
- **C**ommunication & Collaboration
- **E**fficiency & Flow

The world has changed.

Our metrics haven't.

Who wrote this code?

The Identity Problem

Before AI

```
PR #1234
Author: @steipete
```

Simple. One person.

After AI

```
PR #1234
Author: @steipete
  └─ GitHub Copilot (suggestions)
  └─ Claude (refactoring)
  └─ Dependabot (deps)
  └─ Human (review + final edit)
```

Who gets credit?

Your Contribution Graph is Lying

Human

Traditional contributor

Bot

Rule-based automation

dependabot, renovate, codecov

AI-Assisted Human

Copilot, Claude, ChatGPT

Autonomous Agent

AI-driven, goal-oriented

Devin (Cognition), Cursor Agent (Anysphere), Claude Code (Anthropic)

Bots vs Agents: Know the Difference

🤖 **Bots (Rule-Based)** — Triggered by events, deterministic, single-purpose

- dependabot → "update deps"
- renovate → "update deps"
- codecov → "post coverage"

Easy to identify: predictable, repetitive

🦾 **Agents (Autonomous AI)** — Goal-driven, multi-step reasoning, non-deterministic

- Devin (Cognition) → "implement auth flow"
- Cursor Agent (Anysphere) → "refactor module"
- Claude Code (Anthropic) → "fix this bug"

Harder to track: looks like human work

Agents are the hard problem. They do real engineering.

DORA Metrics: The Cracks

Metric	Problem in AI Era
Deployment Frequency	Bots inflate this 10x
Lead Time	Agents ship instantly
Change Failure Rate	Who's accountable? Human or Agent?
Contributor Leaderboard	@dependabot is #1, @devin-ai is #2

We're not measuring developer productivity.

We're measuring **bot activity + agent output + AI token consumption.**

Slop Rate: The Quality Signal

What is Slop Rate?

```
Slop Rate = Closed PRs / Total PRs
```

PRs that were **closed without merging**:

- Rejected by reviewers
- Abandoned by author
- Superseded by better solution
- Failed CI/CD permanently

Why It Matters for AI

Contributor Type	Expected Slop
Human	~5-10%
Bot	~1-2%
Agent	???
AI-Assisted Human	???

High agent slop rate = **poor prompt engineering** or **wrong tool for the job**

Track slop rate by contributor type to measure AI code quality

We need new metrics.

DORA and SPACE got us here. They won't get us there.

ADAPT Metrics

A framework for developer metrics in the AI era

- **A**ttribution — Who wrote this? Human, bot, agent, AI-assisted human
- **D**istribution — Which models and tools are driving your codebase
- **A**utonomy — Agent task completion, human intervention rate, slop rate
- **P**roductivity — Lines changed per 1K tokens, efficiency per contributor type
- **T**oken Economics — Tokens per PR, tokens per feature, cost per change

The Architecture

Orchestration Options

Meltano (this demo)

- Singer-based ELT
- Plugin ecosystem (taps + targets)
- `target-elasticsearch` built for this

Airflow / Dagster

- DAG-based orchestration
- Complex dependencies & scheduling
- Production monitoring & alerting

K8s CronJobs / CI Pipelines

- Kubernetes CronJobs
- GitHub Actions / GitLab CI
- Lightweight, infra you already have

The pipeline pattern matters more than the tool.

Data Model: Denormalized for Elastic

```
{
  "event_type": "commit",
  "timestamp": "2026-03-28T14:32:15Z",
  "contributor": {
    "login": "steipete",
    "type": "human",          // human | bot | agent
    "org": "openclaw",
    "team": "core"
  },
  "traditional_metrics": {
    "lines_added": 47,
    "lines_deleted": 12,
    "time_to_merge_hours": 4.2,
    "review_cycles": 1
  },
  "ai_metrics": {
    "ai_assisted": true,
    "tokens_input": 2847,
    "tokens_output": 1203,
    "model": "claude-sonnet-4-20250514",
    "tool_calls": ["read_file", "edit_file"],
    "session_id": "sess_pr59760_01"
  },
  "embeddings": { "commit_message_vector": [ ... ] }
}
```

Identity Classification

```python
def classify_contributor(login: str) -> str:
    # Bots: rule-based automation
    if login.endswith('[bot]'):
        return 'bot'
    if login in KNOWN_BOTS:      # dependabot, renovate, codecov
        return 'bot'

    # Agents: autonomous AI (the hard problem)
    if login in KNOWN_AGENTS:    # devin-ai, cursor-agent, openclaw-agent
        return 'agent'

    return 'human'

def enrich_event(event, ai_audit_logs) -> dict:
    """Join on user_id + timestamp window"""
    ai_session = find_matching_session(event, ai_audit_logs)

    return {
        **event,
        'ai_assisted': ai_session is not None,
        'ai_tokens': ai_session.tokens_total if ai_session else 0,
        'ai_model': ai_session.model if ai_session else None,
        # Agent-specific
        'autonomous': event.contributor_type == 'agent',
        'required_human_review': ai_session.had_intervention if ai_session else False
    }
```

AI Audit Log Format

```
{
  "event_id": "evt_a1b2c3d4",
  "timestamp": "2026-03-28T14:32:15Z",
  "user_id": "usr_steipete",
  "model": "claude-sonnet-4-20250514",
  "tokens_input": 2847,
  "tokens_output": 1203,
  "session_id": "sess_pr59760_01",
  "tool_calls": ["read_file", "edit_file", "run_terminal"],
  "pr_number": 59760
}
```

Join on `user_id` + timestamp window to enrich GitHub events with AI usage data.

Collecting Session Metrics

You can't analyze what you don't capture.

Tag every PR and commit with the AI session that produced it.

```
# Claude Code embeds session IDs in commits
Co-Authored-By: Claude (session_id: sess_abc123)

# Or tag PRs via labels / metadata
gh pr create --body "ai_session_id: sess_abc123"
```

- Claude Code exposes session IDs — capture them
- Build hooks that auto-tag commits with session context
- Without this link, AI audit logs are **orphaned data**
- The join key between your git history and token usage is the **session ID**

The Transform: DuckDB Denormalization

```sql
CREATE TABLE contribution_events AS
SELECT
    'commit_' || sha AS event_id,
    'commit' AS event_type,
    c.timestamp::timestamp AS timestamp,
    author_login AS contributor_login,
    CASE
        WHEN author_login LIKE '%[bot]' THEN 'bot'
        WHEN author_login IN ('devin-ai', 'cursor-agent') THEN 'agent'
        ELSE 'human'
    END AS contributor_type,
    additions AS lines_added,
    COALESCE(ai.tokens_total, 0) AS ai_tokens_total,
    ai.model AS ai_model,
    ai.tokens_total > 0 AS ai_assisted
FROM raw_commits c
LEFT JOIN (
    SELECT DISTINCT ON(user_email) *
    FROM raw_ai_logs WHERE pr_number IS NULL
    ORDER BY user_email, tokens_total DESC
) ai ON c.author_email = ai.user_email
```

Demo

Build It Yourself

```
# Clone the repo
git clone https://github.com/dtmirizzi/meltano-elastic-presentation

# Start the stack
cd demo && docker-compose up -d

# Run the pipeline
docker-compose exec meltano meltano run tap-github target-duckdb
docker-compose exec meltano meltano run tap-jsonl target-duckdb
docker-compose exec meltano meltano invoke dbt:run
docker-compose exec meltano meltano run tap-duckdb target-elasticsearch

# Open dashboards
open http://localhost:5601
```

All open source. All in Docker.

`target-elasticsearch` → hub.meltano.com/loaders/target-elasticsearch

So What Did We Find?

It was a confluence of issues — not one root cause.

- **RTO policies** decreased morale, engagement dropped
- **GitLab runner migration** led to longer build & deployment times
- **Build fragility** compounded — flaky CI eroded trust in the pipeline

The metrics told the story. Without them, it was all anecdotal.

Why It Matters More Now

Back then we were measuring **humans and their tools**.

Now we're onboarding **autonomous agents** — contributors that:

- Ship code without human initiation
- Don't have morale, but do have failure rates
- Can amplify problems *and* mask them
- Need accountability just like humans do

> If we needed metrics to diagnose a productivity decline across 200 engineers, imagine diagnosing one across **200 engineers + 50 agents**.

Key Takeaways

1. **DORA metrics still matter** — but they need filtering
2. **Identity is complex now** — human, bot, agent, AI-assisted
3. **Token economics are real** — track AI consumption per feature
4. **Denormalize for Elastic** — enrich at ETL
5. **Semantic search is powerful** — embed everything

Your metrics need to evolve with your tools.

dtizzal.com

Slides + Demo Repo

DT Mirizzi

🔌 hub.meltano.com/loaders/target-elasticsearch

Questions?

The lobster way.